

RELATÓRIO TEÓRICO / PEDIDOS VELOZ

LOJA VELOZ / CLOUD DEVOPS

Entrega contínua de uma plataforma de pedidos em microsserviços

Parte teórica - arquitetura cloud-native, DevOps e observabilidade

Aluno: Antonio Silva | Curso: Análise e Desenvolvimento de Sistemas

Instituição: Unifecaf | Tutor: Fernando Melo

Vídeo: link será inserido após a gravação

Microserviços e o papel de DevOps

Uma arquitetura de microserviços divide o sistema em capacidades de negócio pequenas e independentes. Cada serviço expõe um contrato, controla seus próprios dados e pode evoluir sem exigir a publicação conjunta de toda a aplicação. Essa autonomia ajuda equipes diferentes, mas cobra disciplina: falhas de rede passam a fazer parte do fluxo, versões precisam conviver e uma operação sem telemetria perde rapidamente a visão do todo.

DevOps aproxima desenvolvimento e operação por meio de responsabilidade compartilhada e automação. Em vez de uma entrega manual e rara, o código percorre testes, validações de segurança, build de imagem e implantação repetível. O objetivo não é apenas publicar mais rápido, mas tornar cada mudança pequena, observável e reversível. A metodologia 12-Factor reforça essa direção ao separar configuração do código, tratar logs como fluxo e favorecer processos descartáveis e sem estado.

Docker e Kubernetes cumprem papéis diferentes

Docker empacota aplicação, runtime e dependências em uma imagem imutável. O mesmo artefato pode ser executado na máquina do desenvolvedor, no pipeline e no cluster, reduzindo a diferença entre ambientes. Docker Compose organiza vários contêineres locais em um único arquivo, com redes, volumes, variáveis e ordem de inicialização. Ele é adequado para desenvolvimento e testes locais, quando um comando precisa subir todo o conjunto.

Kubernetes atua depois da imagem pronta. Ele mantém réplicas, distribui tráfego, reinicia processos, controla configuração e executa atualizações graduais. Um Deployment descreve o estado desejado, Services criam endereços estáveis, ConfigMaps e Secrets separam parâmetros, e probes indicam se o processo está vivo e se pode receber tráfego. Portanto, Docker resolve empacotamento e execução do contêiner, enquanto Kubernetes resolve coordenação, disponibilidade e escala de muitos contêineres.

Orquestração e entrega contínua

Em um sistema distribuído, a orquestração precisa considerar mais que quantidade de pods. Readiness evita que uma réplica incompleta receba chamadas, liveness reinicia somente um processo realmente travado, e requests e limits dão base ao escalonamento e protegem o nó. O Horizontal Pod Autoscaler observa o uso de recursos e ajusta réplicas. Essa escala horizontal combina com serviços sem estado, mas bancos e filas exigem persistência, backup e uma estratégia própria de alta disponibilidade.

CI/CD estabelece verificações iguais para todas as mudanças. Na integração contínua entram lint, testes, validação dos manifests e busca por vulnerabilidades. A entrega contínua cria imagens versionadas e deixa o deploy disponível sob um gate. Usar a mesma imagem aprovada até produção preserva rastreabilidade. Tags semânticas explicam a versão e o hash do commit identifica exatamente a origem. Secrets do pipeline devem vir do ambiente protegido, nunca do repositório.

A estratégia rolling update foi escolhida para a Loja Veloz porque usa recursos nativos do Kubernetes e mantém a versão anterior enquanto a nova passa pelas probes. Com `maxUnavailable` igual a zero e `maxSurge` igual a um, a troca evita indisponibilidade planejada. Se o rollout não ficar saudável, o pipeline executa rollback. Canary ofereceria controle mais fino de risco, mas exigiria roteamento progressivo e análise automática de métricas que não cabem no MVP de quatro semanas.

Observabilidade: métricas, logs e traces

Monitoramento informa se um limite conhecido foi ultrapassado, enquanto observabilidade ajuda a explicar um comportamento não previsto. Métricas resumem taxa, erro, duração e saturação. Logs estruturados registram fatos com campos pesquisáveis. Traces distribuídos conectam spans de uma mesma operação e mostram onde o tempo foi gasto. Os três sinais se complementam: um alerta aponta aumento de erro, um trace localiza o serviço e o log fornece o detalhe da decisão.

OpenTelemetry evita instrumentação presa a um fornecedor. Na proposta, o contexto W3C atravessa HTTP e cabeçalhos do RabbitMQ, preservando o `trace\_id` do pedido. Prometheus coleta métricas, Loki recebe logs, Tempo armazena traces, e Grafana reúne as consultas. Service mesh não é obrigatório para obter essa correlação. Istio passaria a fazer sentido quando mTLS uniforme, políticas de tráfego e canary no plano de rede fossem requisitos prioritários.

Decisões aplicadas à Pedidos Veloz

Pedidos, Estoque e Pagamentos possuem bancos lógicos separados e trocam eventos pelo RabbitMQ. A criação retorna `202 Accepted`, pois reserva e pagamento continuam em segundo plano. A entrega pelo menos uma vez exige idempotência: cada consumidor registra o identificador recebido. A outbox grava a mudança e o evento na mesma transação, evitando o intervalo em que um pedido poderia ser salvo e seu evento perdido.

Segurança foi tratada em camadas. As imagens usam estágio de build e usuário sem root, enquanto os pods removem capabilities, bloqueiam elevação de privilégio e usam sistema de arquivos somente leitura. O namespace exige Pod Security `restricted`, políticas de rede reduzem comunicação lateral e credenciais são criadas fora do Git. Secrets do Kubernetes ainda precisam de criptografia no etcd e RBAC mínimo em uma implantação real.

A documentação oficial do Kubernetes orientou a configuração de probes, escalabilidade e segurança. A demo oficial do OpenTelemetry complementa a fundamentação ao mostrar instrumentação em várias linguagens e cenários de falha. A Loja Veloz reduz o número de componentes e executa seu ambiente localmente, preservando contratos claros, imagens reproduzíveis e operação guiada por sinais.

Referências primárias

- [1] DOCKER. Multi-stage builds. <https://docs.docker.com/build/building/multi-stage/>
- [2] KUBERNETES. Concepts, probes, HPA e security. <https://kubernetes.io/docs/concepts/>
- [3] THE TWELVE-FACTOR APP. <https://12factor.net/>
- [4] GITHUB. Publishing Docker images. <https://docs.github.com/actions/tutorials/publish-packages/publish-docker-images>
- [5] OPENTELEMETRY. Demo documentation. <https://opentelemetry.io/docs/demo/>
- [6] HASHICORP. Terraform documentation. <https://developer.hashicorp.com/terraform/docs>

Acesso às fontes: 15 ago. 2026.